



Vector DB 개요

{ Vector • Graph 데이터베이스 }

Vector DB — AI 분야의 '떠오르는 별'



NLP의 성장과 벡터화

- ChatGPT 등 NLP 성공의 핵심 기여 요소 = 벡터화(Vectorization)
- 문자를 숫자 배열로 변환하는 기술적 도약
- 컴퓨터가 데이터의 '의미'를 연산 대상으로 인식하기 시작
- 자연어 처리 분야의 급격한 성장을 촉발한 근본 원인



패러다임의 전환

- 단순한 신규 저장소가 아님: 연산의 성격 성격 자체가 다른 별도 제품군
- 기존 DB: '값이 같은 것'을 찾는 도구 (값의 일치)
- Vector DB: '의미가 가까운 것'을 찾는 도구 (의미의 근접성)

정형 데이터와 비정형 데이터의 비중

정형 데이터 (약 20%)

- 📋 주문 정보, 카드 사용 내역 등
- 📁 기존 RDB 환경에서 관리 및 활용 용이

비정형 데이터 (약 80%)

- 🎵 음악, 비디오, 문서, SNS 데이터 등
- 🔗 사전 정의된 형식이 없어 관리와 표현에 난점

기존 시스템의 한계

- 비정형 데이터는 RDB로 저장 및 표현 곤란
- 대부분 단순 저장소에 '쌓아만 두는' 상태로 방치

Vector DB의 해결책

- 데이터의 벡터화로 비정형 간 복잡한 관계 단순 구현
- 방치된 데이터가 '활용 데이터'로 전환되며 사용량 급증 전망

전통 IT 기술과 AI 기술 — 대체가 아닌 공존

상호 보완적 공존 관계

- 두 기술은 추구하는 '품질 기준' 자체가 근본적으로 상이함
- AI가 전통 IT를 대체하는 것이 아닌, 각자의 강점에 맞게 공존할 가능성이 큼



경계선 잡기 — 어디까지 AI인가

정확도형 (100% 정합성 요구)

- 오판 대가가 커서 AI 대체가 곤란한 영역
- 업무 예시: 전자상거래 주문·입금, 회계 마감, 여신 한도 계산, 급여 산정, 규제 보고
- 특이사항: 의료·국방·법조는 90%로 불충분하여 근거 제시형(RAG·지식그래프) 필수

유사도형 (70~90% 수용 가능)

- '고객이 좋아할 상품' 탐색 등 근사치로도 충분한 영역
- 업무 예시: 문서 검색, 챗봇 응대, 이상거래 후보 탐지, 상품 추천

혼합형 (실무 최적 구조)

- AI가 초기 후보를 폭넓게 탐색
- 전통적 규칙(Rule) 엔진이 최종 결과를 확정

**실무 실패의 대부분은 대체 가능 영역과 불가 영역을 구분하지 않은 데서 발생*

Embedding이란?

1단계 : 비정형 데이터 입력

- 단어, 문장, 이미지, 동영상 등 사람이 이해하는 형태의 데이터
- AI 모델은 수학적 함수이므로 텍스트 자체를 입력받을 수 없음



2단계 : 특징 추출 (번역기 역할)

- 임베딩 과정을 통해 데이터 내재된 의미와 특징을 추출
- '비정형 데이터 → 수치 표현'으로 변환하는 번역기 역할 수행

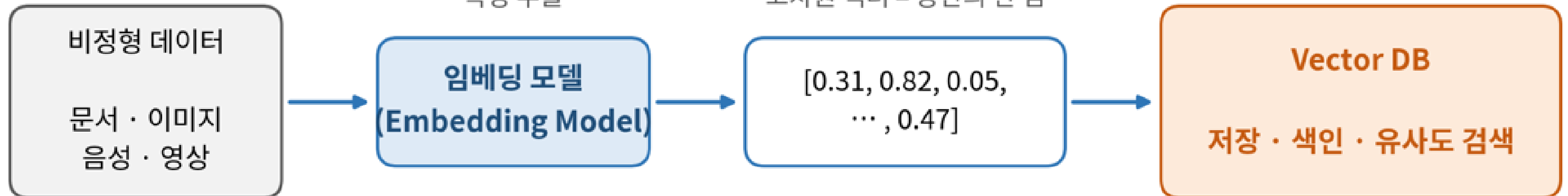


3단계 : 고차원 벡터 숫자 값 출력

- 추출된 특징을 컴퓨터가 계산할 수 있는 숫자 입력/출력값으로 반환
- 숫자 배열 형태로 저장되어 연산의 대상으로 전환됨

특징 추출

고차원 벡터 = 공간의 한 점



임베딩을 만드는 것은 DB가 아니라 '임베딩 모델'

역할의 명확한 분리

- Vector DB: 생성된 벡터 값의 '저장'과 '검색'만 담당
- **임베딩 모델**: 데이터를 수치화하여 벡터를 생성하는 '주체'
- 동일한 데이터라도 적용된 임베딩 모델에 따라 벡터 값과 차원 수가 완전히 달라짐

초기 설계 결정의 중요성

- 서로 다른 모델이 생성한 벡터는 같은 공간에 존재하지 않아 비교(검색) 불가
- 모델 교체 시, 기존 데이터 전체에 대한 **재임베딩** 필수
- 대규모 코퍼스일수록 실질적인 비용과 시간의 막대한 손실 발생
- 핵심: 임베딩 모델 선정은 되돌리기 어려운 가장 중요한 초기 설계 결정

(참고) MTEB 한국어 Retrieval 모델 평균 성능 랭킹 (NDCG@10)

주요 분석 결과

- Sionic AI (comsat-embed-ko-8b-preview) 단독 1위 달성 (0.7930점)
- Qwen3 모델군 강세: 8B(2위), 4B(3위) 사이즈 모두 최상위권 포진
- 국내 기업(Upstage, Sionic 등) 및 글로벌 빅테크(Microsoft) 모델 간 치열한 기술 경쟁 입증
- 1위 모델과 10위 모델 간 성능 편차 약 0.0351점 수준으로 극도의 정밀한 성능 격차 형성

순위	모델	Avg
1	sionic-ai/comsat-embed-ko-8b-preview	0.7930
2	Qwen/Qwen3-Embedding-8B	0.7825
3	Qwen/Qwen3-Embedding-4B	0.7718
4	upstage/solar-embedding-1-large	0.7674
5	microsoft/harrier-oss-v1-27b	0.7669
6	dragonkue/snowflake-arctic-embed-l-v2.0-ko	0.7636
7	codefuse-ai/F2LLM-v2-8B	0.7621
8	nlpai-lab/KURE-v1	0.7603
9	telepix/PIXIE-Rune-v1.5	0.7602
10	nvidia/llama-nemotron-embed-v1-1b-v2	0.7579

Vector 값이란?

공간상의 하나의 '점(Point)'

이미지, 문서, 음악 등 실제 객체와 개념(뉴스 기사, 프로필, 날씨 패턴 등)을
컴퓨터가 이해하는 좌표상의 '점'으로 변환한 숫자 형태의 값

벡터 값의 핵심 속성

- 상대적 의미 부여: 개별 숫자 값 자체는 무의미하며, 의미는 점들 간의 거리·각도 등 상대적 관계에서 발생 (RDB 컬럼 값과의 결정적 차이)
- 의미론적 유사성: 두 점의 근접도를 정량화하여 측정
- 활용 분야: 클러스터링(군집화), 추천 시스템, 데이터 분류에 핵심적으로 활용

'점'이 의미하는 것 — 지도 좌표 비유



2차원: 지도 좌표 비유

- 위도와 경도, 단 2개의 값으로 위치를 완벽히 표현
- 하나의 도시 = 지도 위의 한 점
- 예: 서울 = (37.5, 127.0) / 부산 = (35.2, 129.1)
- 두 도시가 얼마나 가까운가? = 두 점 사이의 물리적 거리로 계산



고차원: 컴퓨터의 언어

- 단 2개의 숫자로도 특정 위치를 명확히 표현 가능
- 1,000개 이상의 숫자 값을 사용하는 고차원 '점'은 훨씬 더 정교하고 방대한 의미를 담음
- 결론: 점은 단순하고 모호한 정보가 아니라, 문자·이미지에 대한 명확한 컴퓨터의 언어

Vector 관계 — 거리(Distance)와 각도(Cosine)

🔗 관계 형성 원리의 차이

- RDB: FK(참조키) 연결을 통해 비즈니스 관계를 사전에 명시적으로 생성
- Vector DB: 관계선 개념 자체가 없음. 내부 알고리즘이 거리와 각도를 계산하여 관계를 스스로 도출(발견)
- 결과: 단 하나의 벡터가 수십에서 수천 개의 유의미한 관계를 동시다발적으로 보유함

구분	관계의 표현 방식	관계의 수	특징
RDB	FK(참조키) 연결선	수 개 ~ 수십 개	사전 정의한 관계만 존재
NoSQL DB	Key-Value 형태의 단순 관계	단순/제한적	스키마 유연, 관계 표현 약함
Vector DB	Distance / Cosine 계산	수천 개 이상	연결선 없이 관계가 '발견'됨

관계가 의미를 담는 원리

벡터 공간의 보존 법칙

단어 간의 논리적·의미적 관계가 벡터 공간 내에서
'위치 관계'로 그대로 보존됨

의미론적 관계

- '음'과 '양'의 거리 $\approx$ '물'과 '불'의 거리
- 대립, 유사성 등의 추상적 관계 자체가 수학적 거리로 명확히 표현됨

문법적 관계

- '살다'와 '살았다'의 거리 $\approx$ '죽다'와 '죽었다'의 거리
- 시제 변환 등 문법적 규칙이 벡터 공간에서 일정한 이동 패턴으로 표현됨

*거리뿐 아니라 각도(Cosine)를 통해서도 정밀한 관계
유추 가능

유사도 척도(Similarity Metric)의 선택



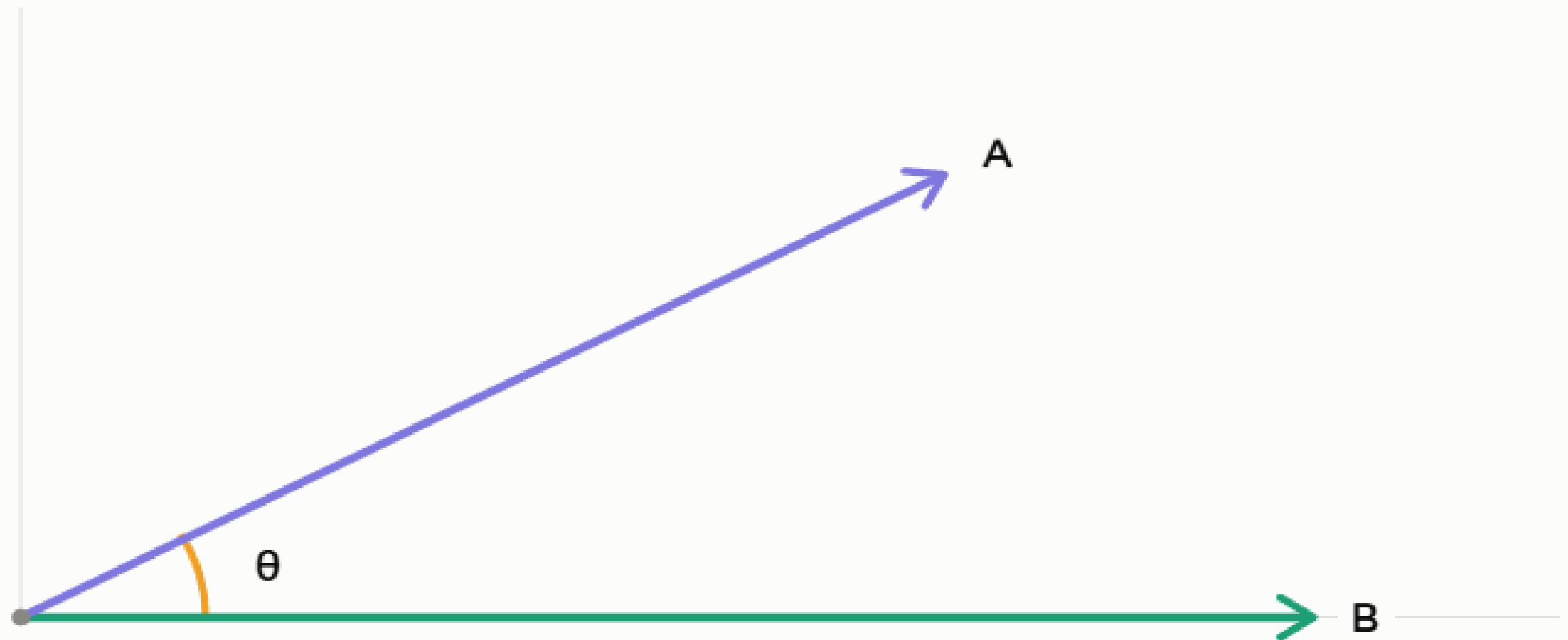
척도 지정 및 설계 시 주의사항

- 컬렉션 생성 시점에 척도 지정이 반드시 필요하며, 임베딩 모델의 학습 방식과 일치시켜야 함
- L2 정규화 적용 시 Cosine과 Dot Product는 순위상 동일한 결과를 출력함
- 대부분 상용 임베딩 모델이 정규화된 벡터를 출력하므로 Cosine을 기본값으로 채택
- **핵심 경고:** 컬렉션 생성 후 척도 변경은 전체 재색인을 요구하므로 반드시 초기 설계 단계에서 확정해야 함

척도	측정 대상	벡터 크기(norm) 영향	주 사용처
Cosine Similarity	두 벡터가 이루는 각도	받지 않음 (방향만 비교)	문서·문장 의미 검색의 표준
Dot Product (내적)	방향 + 크기	받음	정규화 모델, 추천 시스템
Euclidean (L2)	두 점 사이의 직선 거리	받음	이미지 특징, 클러스터링

유사도 척도(Similarity Metric)의 선택

Cosine Similarity — 두 벡터가 이루는 각도 θ 만 봅니다.

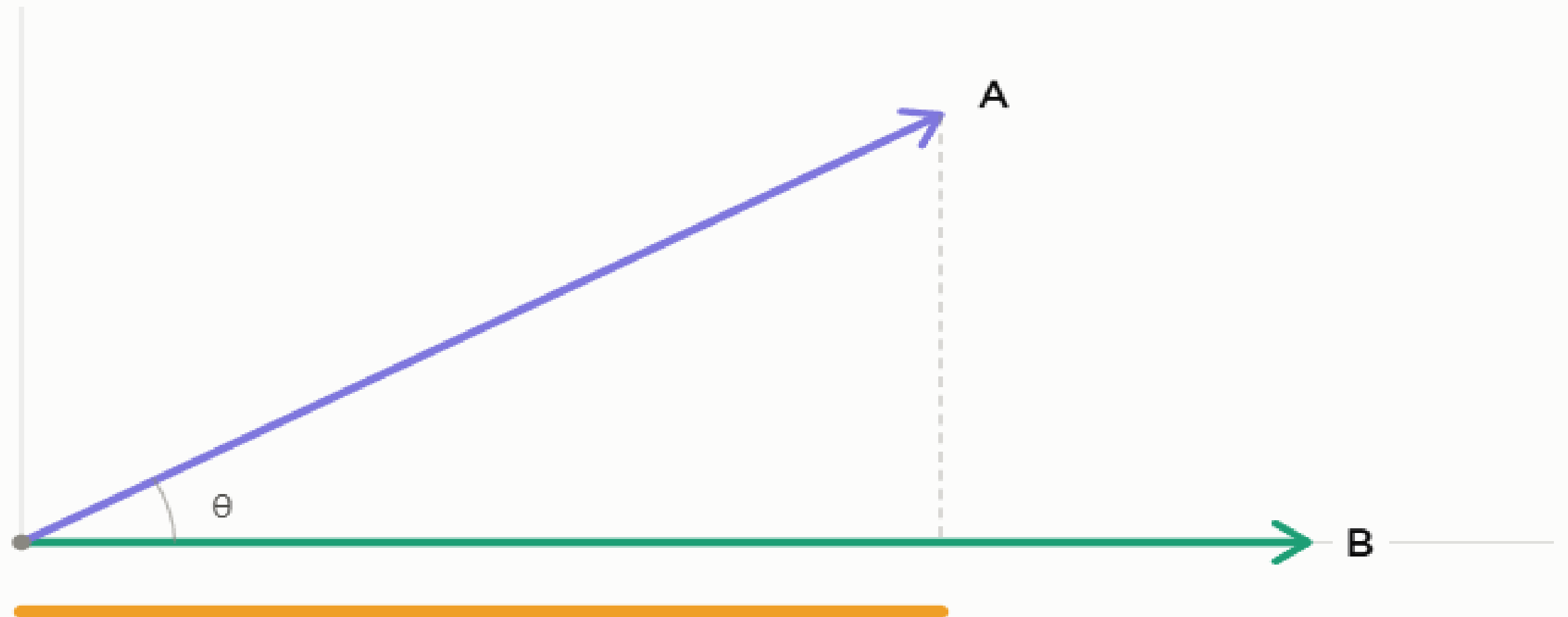


$$\cos \theta = (A \cdot B) / (\|A\| \|B\|)$$

1 = 같은 방향 · 0 = 직교 · -1 = 반대 방향 (길이 무관)

유사도 척도(Similarity Metric)의 선택

Dot Product — A를 B에 투영한 길이에 $\|B\|$ 를 곱합니다. 각도와 길이가 모두 반영됩니다.

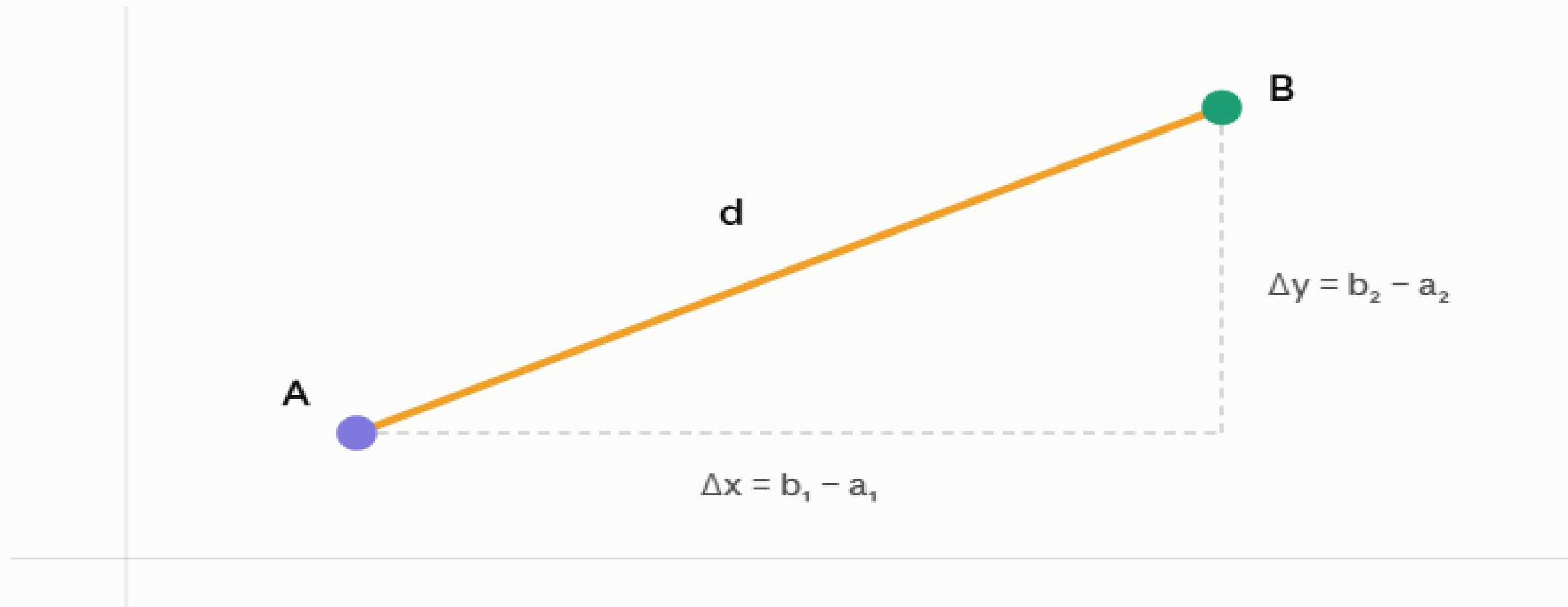


$$\text{투영 길이} = \|A\| \cos \theta$$

$$A \cdot B = \|A\| \|B\| \cos \theta \text{ (길이가 길수록 값이 커짐, 상한 없음)}$$

유사도 척도(Similarity Metric)의 선택

Euclidean (L2) — 벡터를 점으로 보고 두 점 사이의 직선 거리를 잽니다. 작을수록 유사합니다.



$$d(A, B) = \sqrt{\sum (a_i - b_i)^2} \rightarrow 0 \text{이면 동일, 작을수록 유사}$$

Vector DB 및 주요 DBMS 비교 분석

- RDB: '=' 및 'LIKE' 연산자 기반 문자열 일치 및 포함 판단에 국한
- Vector DB: 별도 개발 없이 텍스트·이미지·오디오 등 비정형 데이터를 임베딩(수치화)하여 유사도 기준 정보 획득 가능

RDB

- **핵심 역할:** 정형 데이터 및 트랜잭션 관리
- **강점:** 복잡한 비즈니스 관계의 정확한 표현
- **대표 제품:** Oracle, MySQL, PostgreSQL



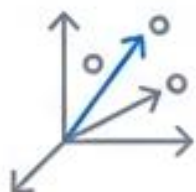
NoSQL DB

- **핵심 역할:** 빅데이터 분산 관리
- **강점:** 대용량 데이터의 유연한 확장 관리
- **대표 제품:** MongoDB, DynamoDB, Redis



Vector DB

- **핵심 역할:** 데이터 유사성 관리
- **강점:** 벡터 기준의 다양한 유사도 관계 관리
- **대표 제품:** Weaviate, Pinecone, Milvus

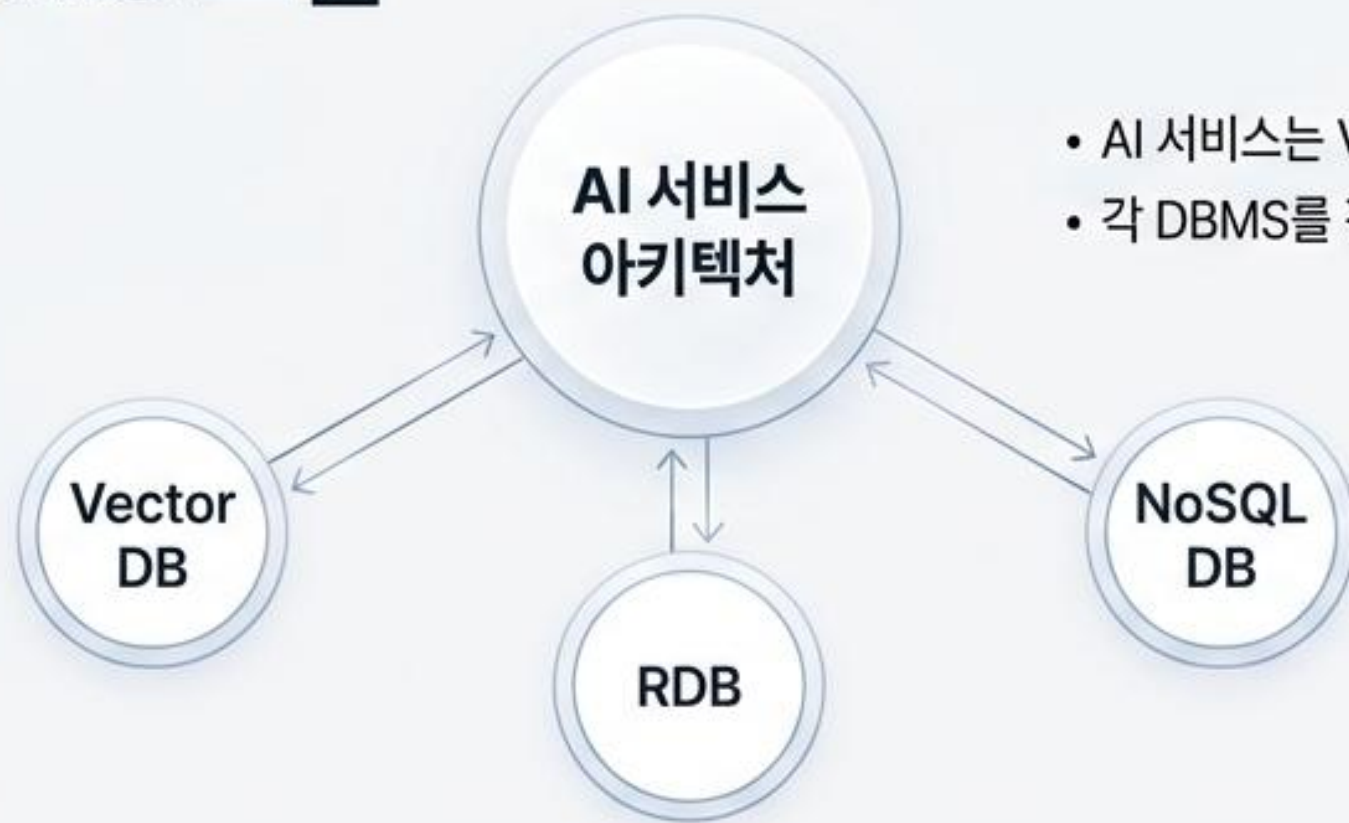


Graph DB

- **핵심 역할:** 개체 간 관계 및 지식 관리
- **강점:** 명시적 관계 탐색 및 다중 홉 추론
- **대표 제품:** Neo4j, Azure Cosmos DB



실제 AI 아키텍처 구성: 다중 DBMS 조합



- AI 서비스는 Vector DB 단독 사용 불가, RDB 및 NoSQL과의 병용 필수
- 각 DBMS를 강점이 가장 잘 발휘되는 최적 지점에 배치하는 것이 원칙

NoSQL DB / 파일 시스템

대용량 원천 데이터
저장 및 관리

Vector DB

임베딩된 Vector 값 저장 및
유사도 검색 수행

RDB

검색 문장, 결과 문서, 대화 이력
등 메타데이터 정밀 관리

핵심 인사이트

- 지식그래프 활용 사례: 팔란티어(Palantir), Insilico Medicine
- 결론: Vector DB와 Graph DB는 배타적 선택이 아닌 강력한 보완 관계 형성

DBMS와 IT 발전의 역사적 상관관계

핵심 분석 및 인사이트

- IT 기술의 급속한 발전 이면에는 항상 DBMS의 근본적 진화가 동반됨
- 데이터 형태의 변화(정형 → 확장성 요구 → 비정형)에 따라 특화 DB 등장

1979년

RDB 출시 (Oracle 등)

- 특징: 정보공학 및 트랜잭션 관리의 중심
- 주요 대상: 정형 데이터

2007년

NoSQL 등장 (Amazon Dynamo 등)

- 특징: 대용량 데이터의 유연하고 확장 가능한 기능 제공
- 주요 대상: 확장성 최우선 데이터

2019년

Vector DB 상용화 (Weaviate 등)

- 특징: LLM 및 NLP(자연어처리) 성능 극대화
- 주요 대상: 비정형 데이터 기반 유사도 탐색

[2026 트렌드 ①] 시장 성숙과 '기능으로서의 벡터'

시장 변화 핵심 요약

- 2022년 이후 급성장 시기 종료, 2026년 현재 과열기를 지나 성숙 단계 진입 완료
- 핵심 변화: 독립 제품이던 벡터 검색이 기존 DB의 한 기능(Vector as a Feature)으로 빠르게 흡수 중
- DBMS 선택 추가 기준:
 - 이미 쓰는 DB로 충분한가?
 - 벡터 검색이 핵심 워크로드인가?

범용/표준 영역

PostgreSQL + pgvector

- 수천만 벡터 이하 규모에서 사실상 표준 기술로 안착
- 운영 대상 시스템이 하나로 축소되어 관리 효율 극대화

엔터프라이즈 통합

기존 RDB 내장화

- 오라클 AI Vector Search 등, 데이터 계층 자체에 벡터 기능 통합

하이엔드 전용 영역

전용 벡터 DB의 입지

- 수억 벡터 이상, 고QPS, 멀티테넌시, 세밀한 필터링 환경에서 확고한 우위 유지 (시장 소멸이 아닌 분화)

[2026 트렌드 ②] 임베딩 모델의 진화와 선택 기준

품질의 본질

벡터의 품질이 곧 검색의 품질 결정
(DB 선택보다 모델 선택의 영향력이 다수)

Matryoshka 표현 학습(MRL)

중요 정보를 앞쪽 차원에 집중.
차원 축소(예: 3,072 → 256) 시에도 품질 유지.
저장 용량 및 검색 비용 대폭 절감 달성
(차원 수가 설계자 조정 값으로 전환)

멀티모달 및 오픈소스

텍스트·이미지 동일 공간 임베딩 지원.
표·그림을 이미지째 검색 가능.
금융·의료 등 온프레미스 환경용
오픈소스 경쟁력 확보

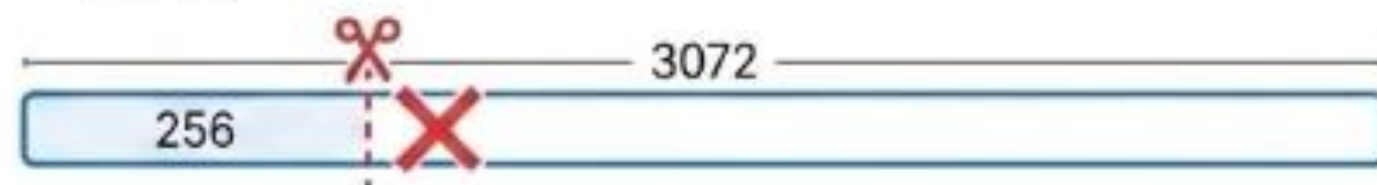
선택 기준	점검 질문	판단 시 주의점
도메인 적합성	전문 용어를 잘 구분하는가	공개 벤치마크보다 자체 평가셋이 우선
언어	한국어 검색 품질이 충분한가	영어 중심 점수는 한국어 성능 보장 불가
차원 수	비용과 품질의 균형점은	MRL 지원 시 차원 축소로 조정 가능
배포 형태	API vs 자체 호스팅	데이터 반출 규제·비용·지연시간 병행 검토
안정성	모델 폐기 시 대응은	재임베딩 비용 사전 산정 필수

Matryoshka 표현 학습(MRL) 핵심 원리 및 기존 임베딩 비교 분석

일반 임베딩의 한계 (Limitation)



- 3,072개 전체 차원이 하나의 덩어리로 정보와 의미를 분산하여 공유
- 앞부분(예: 256개 차원)만 추출 시, 의미가 무작위로 손상되어 검색 품질 급감



MRL 핵심 개념 (Concept)



- 하나의 임베딩 벡터를 앞부분만 절단하여 사용해도 유효한 성능 유지
- 러시아 마트료시카 인형처럼 큰 벡터 내부에 작은 벡터가 겹겹이 내포된 구조로 학습



다중 손실 함수 메커니즘 (Multi-Loss Mechanism)



- 모델 학습 시, 단일 길이가 아닌 다양한 차원 길이에 대해 손실 함수(Loss Function) 동시 적용

$$\mathcal{L} = \mathcal{L}(\text{앞 } 64) + \mathcal{L}(\text{앞 } 128) + \mathcal{L}(\text{앞 } 256) + \dots + \mathcal{L}(\text{전체 } 3,072)$$

정보의 자율적 재배치 (Information Restructuring)



- 다중 손실 함수로 인해 앞쪽 차원만으로 정답을 도출해야 한다는 학습 압박(Pressure) 발생
- 모델 스스로 가장 핵심적인 정보는 앞 차원에, 세부적이고 지엽적인 정보는 뒤 차원에 배치하도록 최적화



[2026 트렌드 ②] 임베딩 모델의 진화와 선택 기준

품질의 본질

벡터의 품질이 곧 검색의 품질 결정
(DB 선택보다 모델 선택의 영향력이 다수)

Matryoshka 표현 학습(MRL)

중요 정보를 앞쪽 차원에 집중.
차원 축소(예: 3,072 → 256) 시에도 품질 유지.
저장 용량 및 검색 비용 대폭 절감 달성
(차원 수가 설계자 조정 값으로 전환)

멀티모달 및 오픈소스

텍스트·이미지 동일 공간 임베딩 지원.
표·그림을 이미지째 검색 가능.
금융·의료 등 온프레미스 환경용
오픈소스 경쟁력 확보

선택 기준	점검 질문	판단 시 주의점
도메인 적합성	전문 용어를 잘 구분하는가	공개 벤치마크보다 자체 평가셋이 우선
언어	한국어 검색 품질이 충분한가	영어 중심 점수는 한국어 성능 보장 불가
차원 수	비용과 품질의 균형점은	MRL 지원 시 차원 축소로 조정 가능
배포 형태	API vs 자체 호스팅	데이터 반출 규제·비용·지연시간 병행 검토
안정성	모델 폐기 시 대응은	재임베딩 비용 사전 산정 필수

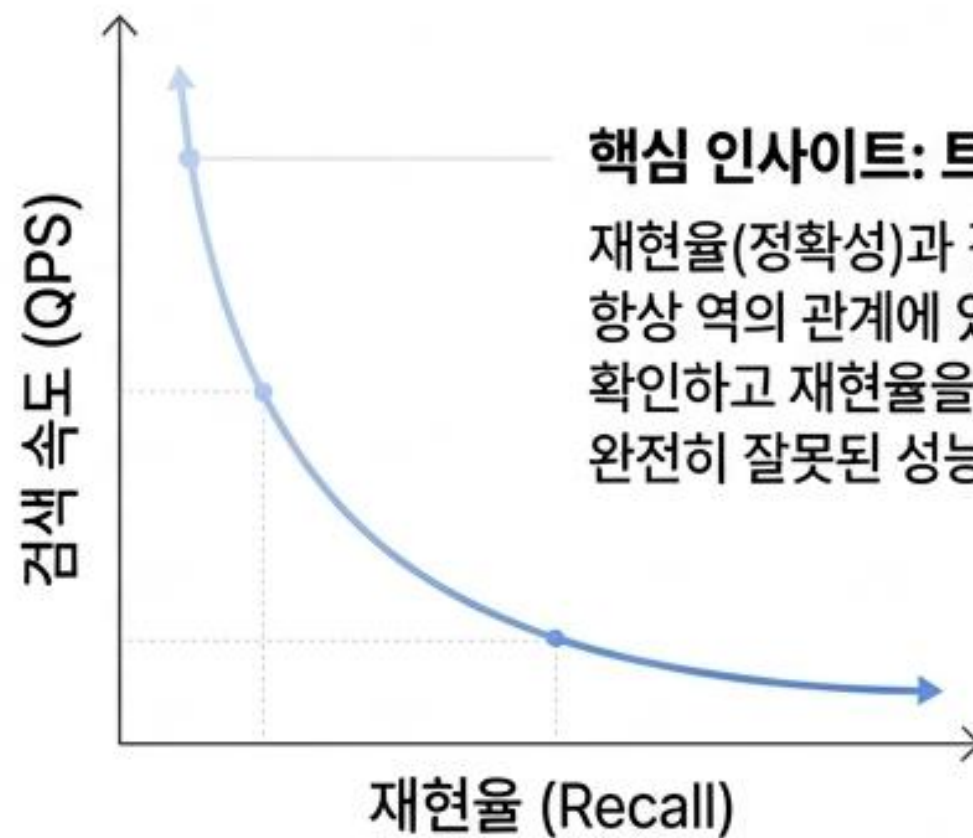
[2026 트렌드 ③] ANN 인덱스의 핵심 원리: 속도와 재현율

벡터 검색의 본질과 한계

- **본질:** 질의 벡터와 가장 가까운 k개의 이웃을 탐색하는 과정
- **전수 비교(Brute-force)의 한계:**
100% 정확도 보장하나 데이터 급증 시 컴퓨팅 파워 감당 불가
- **해결책:** ANN (Approximate Nearest Neighbor, 근사 최근접 이웃) 인덱스 알고리즘 전면 도입

개념 정의: 재현율 (Recall)

정답 이웃 k개 중 실제 알고리즘이 찾아낸 비율



핵심 인사이트: 트레이드오프

재현율(정확성)과 검색 속도(QPS)는 항상 역의 관계에 있음. QPS 수치만 확인하고 재현율을 검증하지 않으면 완전히 잘못된 성능 비교 오류 발생.

주요 ANN 인덱스 유형별 특징 비교

ⓘ 주의사항: 차원 수, 유사도 척도, 인덱스 종류는 컬렉션 설계 시 동시 결정해야 하는 핵심 항목. 사후 변경이 거의 불가하며, 변경 시 재색인(Re-indexing) 필수.

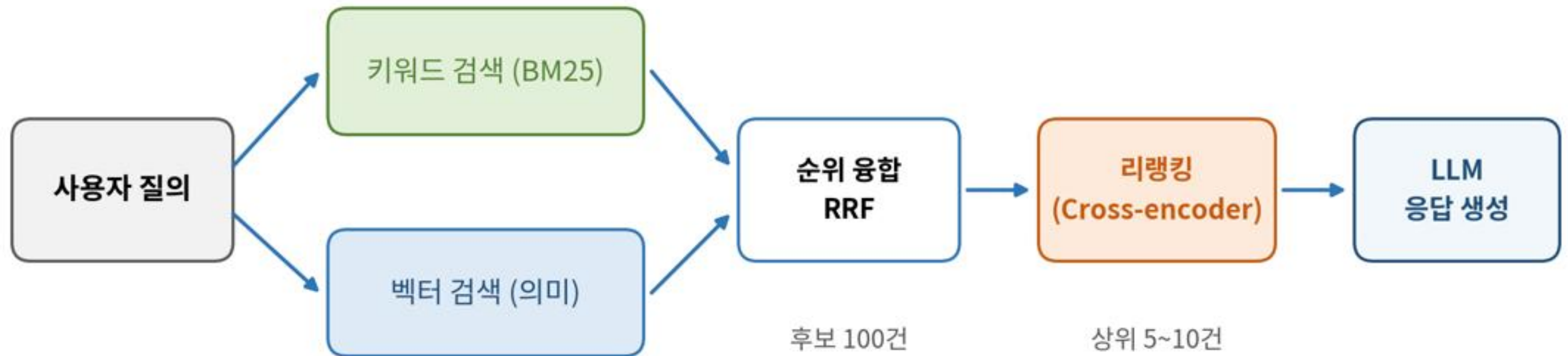
인덱스	원리	장점	유의점
Flat (Brute-force)	전수 비교	100% 정확	데이터가 커지면 사용 불가
IVF	군집으로 나눈 뒤 일부만 탐색	메모리 효율적, 대용량 적합	군집 경계의 이웃을 놓칠 수 있음
HNSW	계층적 그래프 탐색	속도·재현율 균형 최우수 (표준)	메모리 사용량이 큼
PQ / 양자화	벡터를 압축 저장	메모리 대폭 절감	압축 손실로 재현율 하락
DiskANN 계열	디스크 기반 인덱스 탐색	초대용량을 저비용 처리	지연시간이 상대적으로 김

주요 ANN 인덱스 유형별 특징 비교

ANN 인덱스의 속도-재현율 트레이드오프 (개념도)



[2026 트렌드 ④] 하이브리드 검색 및 재순위화 파이프라인



[2026 트렌드 ④] 하이브리드 검색 및 재순위화 파이프라인

핵심 철학: 임베딩은 손실(Lossy) 과정이므로 정확 일치(Exact Match)에 취약.

따라서 2026년 실무 표준은 '빠른 검색으로 후보를 넓게 뽑고, 느린 모델로 좁게 정제한다' 입니다.

단계 1: 하이브리드 검색 (Hybrid Search)

- 키워드 검색(BM25)과 벡터 유사도 검색 동시 수행
- RRF(Reciprocal Rank Fusion) 알고리즘을 통한 검색 순위 융합으로 초기 후보군(약 100건) 도출



단계 2: 재순위화 (Re-ranking)

- 초기 도출된 후보를 교차 인코더(Cross-encoder)로 정밀 재채점
- 가장 적합한 상위 5~10건의 결과로 최종 압축 및 정제



단계 3: 생성 (Generation)

- 엄격하게 선별되고 검증된 근거 데이터만 LLM(대형언어모델)에 전달하여 응답 생성



벡터 단독 검색이 실패하는 전형적 시나리오

실패 원인 및 실무 대응 전략

- **취약점 1:** 고유 번호, 사내 약어 등 임베딩 모델이 사전 학습한 적 없는 생소한 토큰 처리 불가
- **취약점 2:** 부정 표현이 벡터 공간 내에서 긍정 표현과 유의미한 거리 차이로 명확히 분리되지 않음
- **대응 전략:** 해당 유형이 잦은 비즈니스 도메인일수록 키워드 검색 및 메타데이터 필터링 비중을 상향 조정 필수

질문: 계약서 A-2024-0317의 위약금 조항을 찾아줘

실패 이유: 'A-2024-0317'이라는 고유 문서 번호 의미가 전체 임베딩 과정 중 벡터 내에서 희석됨

질문: 현재 재고가 없는 상품 목록을 보여줘

실패 이유: '없음'이라는 부정 표현이 수치화된 벡터 공간에서 잘 구분되지 않아 오작동 발생

[2026 트렌드 ⑤] **Agentic RAG** 및 Context Engineering

Agentic RAG

- 검색 과정을 에이전트의 판단 루프 내부에 배치
- 검색 근거 부족 시, 시스템이 스스로 질의를 재작성하여 재검색 수행 (다중 홉 추론에 매우 효과적)

Graph RAG

- 질문의 답이 데이터 간의 '관계' 자체에 존재할 경우 지식그래프(Knowledge Graph) 결합
- Vector DB와 Graph DB의 이상적인 보완적 결합 모델

Adaptive 라우팅

- 단순 질의는 기본 검색으로, 복잡한 질의만 에이전트 루프로 동적 할당
- 비용(토큰 사용량) 및 지연 시간(Latency)의 효율적 통제 목적

Context Engineering

- RAG를 단순 검색을 넘어 무엇을, 얼마나, 어떤 순서로 보여줄 것인가의 종합 설계 문제로 확장
- 주의점: 컨텍스트 창 확대가 RAG의 불필요를 의미하지 않음. 전량 투입은 비용/지연 선형 증가 및 권한 필터링 불가를 야기하므로 정제된 검색은 필수 불가결함

Vector DB 도입 시 5대 필수 리스크 고려사항

영역	리스크 (Risk)	실무 대응 방향 (Mitigation)
모델 종속성	임베딩 모델 교체 시 전체 데이터의 재임베딩 불가피	프로젝트 초기 단계부터 재임베딩 소요 비용 및 절차를 아키텍처 설계에 의무 포함
보안 및 권한	벡터 자체에 원문 의미가 담겨 있어 권한 없는 사용자에게 노출될 위험 존재	메타데이터 기반의 권한 필터를 벡터 검색 시점에 강력하게 강제 적용
개인정보 보호	임베딩 데이터로부터 원문을 부분적으로 복원해 내는 '역전 공격' 위험 노출	민감정보는 사전에 마스킹 처리 후 임베딩하고, 저장 시 반드시 암호화 적용
품질 평가	객관적 지표 없이 담당자 체감에만 의존한 주관적이고 부정확한 운영	자체 평가셋 구축 후 Recall(재현율) 및 nDCG 등 정량 지표 기반 필수 관리
비용 증가	고차원 및 대용량 인덱스 운용 시 메모리 비용의 기하급수적 급증	차원 축소 모델(MRL) 도입, 양자화 기법 활용, 계층적 저장소 설계 적용 등 최적화 이행